

```

1 begin
2   using Oscar: ZZ, QQ, elem_type, factor, residue_ring, unit_group, gcd, lcm,
      polynomial_ring, gen, abelian_group, sylow_subgroup, ideal, matrix_space,
      exponent, coeff
3   using Latexify
4 end

```

i

```

1 begin
2   # Global Ring/Field Bindings
3   Z = ZZ
4   Q = QQ
5   fmpz = elem_type(Z)
6
7   # Gaussian Ring Setup via Quotient Ring  $Z[x]/(x^2 + 1)$ 
8   R_poly, x = polynomial_ring(ZZ, "i")
9   G, _ = residue_ring(R_poly, x^2 + 1)
10  i = gen(G)
11 end

```

Lecture Ten: The Structure of Numbers

Scope

This lecture dives deeper into the **algebraic structure** of the integers $\mathbb{Z}$ and related number systems, exploring how **arithmetic properties** emerge from underlying **mathematical frameworks**.

I. Integers as a Ring

Integers form a commutative ring with identity.

Integer Addition Proof:

- **Pure FP:** 60
- **Abstract Algebra:** 60
- **Group Theory:** Abelian group element [60]
- **Mathematical Proof Aggregation:** true

All paradigms correspond structurally.

```
1 begin
2   # 1. Pure FP (Stateless)
3   ring_add_fp(a::fmpz=Z(48), b::fmpz=Z(12)) = a + b
4
5   # 2. Abstract Algebra (Oscar)
6   function ring_add_aa(a::fmpz=Z(48), b::fmpz=Z(12))
7     return Z(a) + Z(b)
8   end
9
10  # 3. Group Theory (Symmetric/Cyclic properties)
11  function ring_add_group(a::fmpz=Z(48), b::fmpz=Z(12))
12    # Addition in Z is the action of the infinite cyclic group (Z, +)
13    A = abelian_group([0])
14    a_elem = a * A[1]
15    b_elem = b * A[1]
16    return a_elem + b_elem
17  end
18
19  ans_ring_fp = ring_add_fp()
20  ans_ring_aa = ring_add_aa()
21  ans_ring_grp = ring_add_group()
22
23  Markdown.parse("""
24  ### Integer Addition Proof:
25  * **Pure FP**: $(ans_ring_fp)
26  * **Abstract Algebra**: $(ans_ring_aa)
27  * **Group Theory**: $(ans_ring_grp)
28  * **Mathematical Proof Aggregation**: $(ans_ring_fp == ans_ring_aa ==
29    ans_ring_grp[1])
30
31  All paradigms correspond structurally.
32  """)
33 end
```

II. Integral Domains and UFDs

$$\mathbb{Z}$$

is a UFD; every element factors uniquely into prime atoms.

Prime Factorization Proof (48):

- **Pure FP:** Nemo.ZZRingElem[2, 2, 2, 2, 3]
- **Abstract Algebra:** $2^4 * 3$
- **Group Theory (Sylows):** Hecke.FinGenAbGroup[Z/16, Z/3]
- **Mathematical Proof Aggregation:** true

```
1 begin
2   # 1. Pure FP (Recursive Factorization)
3   function factor_fp(n::fmpz=Z(48))
4       n <= Z(1) && return fmpz[]
5       p = first(Iterators.filter(d -> mod(n, d) == Z(0), 2:n))
6       return vcat([fmpz(p)], factor_fp(div(n, fmpz(p))))
7   end
8
9   # 2. Abstract Algebra (Oscar Factorization)
10  factor_aa(n::fmpz=Z(48)) = factor(n)
11
12  # 3. Group Theory
13  # Factorization determines the decomposition of cyclic group Z_n into Sylow p-
14  # subgroups (by Chinese Remainder Theorem)
15  function factor_group(n::fmpz=Z(48))
16      A = abelian_group([n])
17      return [syLOW_subgroup(A, p)[1] for (p, _) in factor(n)]
18  end
19
20  ans_factor_fp = factor_fp()
21  ans_factor_aa = factor_aa()
22  ans_factor_grp = factor_group()
23
24  Markdown.parse("""
25  ### Prime Factorization Proof (48):
26  * **Pure FP**:: $(ans_factor_fp)
27  * **Abstract Algebra**:: $(ans_factor_aa)
28  * **Group Theory (Sylows)**:: $(ans_factor_grp)
29  * **Mathematical Proof Aggregation**:: $(length(unique(ans_factor_fp)) ==
30    length(ans_factor_aa) == length(ans_factor_grp))
31  """)
32 end
```

III. Units and Divisibility

The divisibility poset relies on GCD/LCM structures.

Divisibility and LCM Proof:

- Pure FP: 48
- Abstract Algebra (Ideal Intersection): 48
- Group Theory (Direct Product Exponent): 48
- Mathematical Proof Aggregation: true

```
1 begin
2   # 1. Pure FP (Logic modeling)
3   lcm_fp(a::fmpz=Z(48), b::fmpz=Z(12)) = abs(a * b) ÷ gcd(a, b)
4
5   # 2. Abstract Algebra
6   function lcm_aa(a::fmpz=Z(48), b::fmpz=Z(12))
7     # LCM is algebraically the intersection of two principal ideals
8     I1 = ideal(ZZ, a)
9     I2 = ideal(ZZ, b)
10    I3 = intersect(I1, I2)
11    return gen(I3)
12  end
13
14  # 3. Group Theory View (Exponent of Direct Product)
15  function lcm_group(a::fmpz=Z(48), b::fmpz=Z(12))
16    # The exponent of the abelian group  $Z_a \times Z_b$  geometrically maps to their LCM
17    A = abelian_group([a, b])
18    return exponent(A)
19  end
20
21  ans_lcm_fp = lcm_fp()
22  ans_lcm_aa = lcm_aa()
23  ans_lcm_grp = lcm_group()
24
25  Markdown.parse(""
26    ### Divisibility and LCM Proof:
27    * **Pure FP**: $(ans_lcm_fp)
28    * **Abstract Algebra (Ideal Intersection)**: $(ans_lcm_aa)
29    * **Group Theory (Direct Product Exponent)**: $(ans_lcm_grp)
30    * **Mathematical Proof Aggregation**: $(ans_lcm_fp == ans_lcm_aa == ans_lcm_grp)
31    "")
32 end
```

IV. Extension: Gaussian Integers and Modular Rings

Gaussian Integers Proof:

- **Pure FP (Pair ADT):** (-1, 5)
- **Abstract Algebra (Gaussian Ring):** $5^*i - 1$
- **Group Theory (Matrix Action):** (-1, 5)
- **Mathematical Proof Aggregation:** true

```
begin
  # 1. FP Data Modeling (Pair ADT)
  gaussian_mul_fp(z1::Tuple{fmpz, fmpz}=(Z(1), Z(1)), z2::Tuple{fmpz, fmpz}=(Z(2),
  Z(3))) =
    (z1[1]*z2[1] - z1[2]*z2[2], z1[1]*z2[2] + z1[2]*z2[1])

  # 2. Abstract Algebra (Oscar Gaussian Ring)
  function gaussian_mul_aa(a::fmpz=Z(1), b::fmpz=Z(1), c::fmpz=Z(2), d::fmpz=Z(3))
    g_elem1 = G(a + b*i)
    g_elem2 = G(c + d*i)
    return g_elem1 * g_elem2
  end

  # 3. Group Theory (Gaussian Action)
  function gaussian_mul_group(a::fmpz=Z(1), b::fmpz=Z(1), c::fmpz=Z(2),
  d::fmpz=Z(3))
    # The multiplicative group of Gaussian integers acts as rotations and
    dilations
    # represented strictly by matrix multiplication in GL(2, Z)
    MS = matrix_space(ZZ, 2, 2)
    M1 = MS([a -b; b a])
    M2 = MS([c -d; d c])
    res = M1 * M2
    return (res[1, 1], res[2, 1])
  end

  ans_gauss_fp = gaussian_mul_fp()
  ans_gauss_aa = gaussian_mul_aa()
  ans_gauss_grp = gaussian_mul_group()

  Markdown.parse("""
  ### Gaussian Integers Proof:
  * **Pure FP (Pair ADT)**: $(ans_gauss_fp)
  * **Abstract Algebra (Gaussian Ring)**: $(ans_gauss_aa)
  * **Group Theory (Matrix Action)**: $(ans_gauss_grp)
  * **Mathematical Proof Aggregation**:: $(ans_gauss_fp == (coeff(ans_gauss_aa.data,
  0), coeff(ans_gauss_aa.data, 1)) == ans_gauss_grp)
  """)
end
```

B. Polynomial Rings $\mathbb{Z}[x]$

- Polynomials with integer coefficients
- Also a UFD
- Links to generating functions (used in Lecture 8 via Oscar.jl)

C. Modular Rings $\mathbb{Z}/m\mathbb{Z}$

- Quotient ring—finite structure
- Not always a domain (e.g., $2 \cdot 3 \equiv 0 \pmod{6}$)
- Units form group $(\mathbb{Z}/m\mathbb{Z})^\times \rightarrow$ group theory application

V. Computational Implications

A. Data Representation

- Integer structures inform exact computation in data pipelines
- Avoids floating-point errors in discrete analytics

B. Symbolic Algebra in Julia

- Oscar.jl enables:
 - Ring operations
 - Ideal membership
 - Factorization across domains
- Enables proof-by-computation in your notebooks

C. Functional Modeling of Structures

- ADT-based design → aligns with FP-DOD

VI. Connections to Discrete Mathematics

Concept	Discrete Math Role	CS/Data Application
Rings	Algebraic foundation	Hashing, coding theory

VII. Bridge to Future Lectures

- Prepares for:
 - Lecture 11: Modular Arithmetic (concrete ring $\mathbb{Z}/m\mathbb{Z}$)
 - Lecture 12: Public Key Cryptography (RSA over $\mathbb{Z}$)
 - Lecture 17–22: Graph Theory (algebraic graph invariants)

VIII. Methodology Recap

Paradigm	Oscar / Computational Math Approach
Pure FP	Lazy Iterators (<code>Iterators.filter</code>), ADTs (<code>Tuples/Pairs</code>), Stateless Functions
Abstract Algebra	Rings (<code>ZZ</code> , <code>polynomial_ring</code>), <code>residue_ring</code> , ideal generation and <code>intersect</code>
Group Theory	<code>abelian_group</code> (cyclic mappings), <code>sylow_subgroup</code> , <code>matrix_space</code> ($GL(2,\mathbb{Z})$)

All numerical derivations map structurally perfectly across mathematical paradigms.